Proyecto 🦙Chat (LlamaChat)

Sistemas Distribuidos

Diego Sevilla Ruiz

Murcia, 2026

Índice

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

1. Introducción

Introducción

- Estas transparencias describen el proyecto de prácticas de Sistemas Distribuidos
- Constará del desarrollo de una **aplicación distribuida formada por un conjunto de servicios**:
  1. gestión de usuarios y autenticación (JWT),
  2. estadísticas de acceso y monitorización (Prometheus),
  3. servicio de sesión de *prompt* LlamaChat,
  4. servicio de log de sesiones de *prompt*,
  5. pruebas End-to-End (E2E) automáticas
- Se hará uso de las tecnologías REST, gRPC, Kafka, Docker, Prometheus y Selenium

2. Especificación del proyecto

Introducción

Objetivo

El objetivo del proyecto es escribir una aplicación distribuida compuesta de varios contenedores Docker que ofrecen la funcionalidad de gestionar **usuarios**, **estadísticas de acceso y uso del servicio**, **sesiones de *prompt***, **log de sesiones** y **validación automática del interfaz** (E2E).

- Los servicios se desarrollarán con REST y con gRPC
- Cada servicio se instalará en un contenedor independiente
- Se tendrán que diseñar los APIs de los servicios en gRPC y los servicios REST que corresponda
- La seguridad es fundamental: se usará **JWT (JSON Web Tokens)** para todas las autenticaciones entre servicios y clientes

Capas de la aplicación

La aplicación se dividirá en las tres capas estándar de las aplicaciones distribuidas:

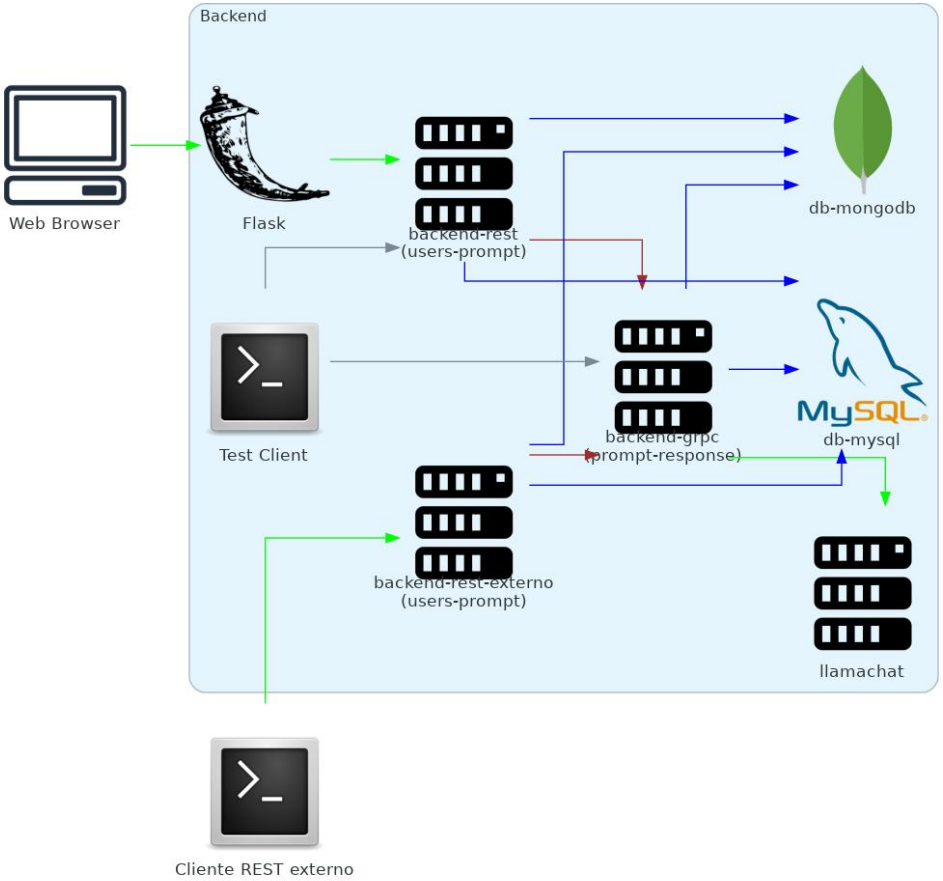
Frontend (F) El *frontend* se ofrecerá a través de un acceso Web. Se podrá implementar usando tecnologías de cliente como Bootstrap[1]. Y la parte servidor, que formará parte del *backend*, se podrá implementar haciendo uso de cualquier tecnología, desde PHP hasta Node.js, pasando por Python Flask (veremos un ejemplo)

Backend (B) El *backend* incluye: (1) los servidores relativos a la vista; y (2) los servidores que implementan los servicios REST y gRPC en Java

Datos (D) La capa de datos utilizará una o varias bases de datos para almacenar los datos de todos los servicios

[1] `https://getbootstrap.com/`.

Arquitectura de la aplicación



Sistemas Distribuidos 2023-24

Frontend

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

Frontend

- El *frontend* debe incluir un interfaz web para, **al menos**, realizar las siguientes tareas:
  1. Ofrecer la posibilidad de registro de usuarios
  2. Ofrecer una pantalla de login de usuarios (obteniendo el JWT)
  3. Ofrecer un interfaz para iniciar una sesión de *prompt*, que permita mantener una conversación y terminarla (cada conversación se guardará en el *log*)
  4. Ofrecer un interfaz de administración de *logs* de conversaciones, que permita mostrarlos y eliminarlos
  5. Ver las estadísticas de acceso (obtenidas del sistema de monitorización)
- En el *frontend*, se valorará que sea funcional más que que sea bonito

Backend

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

Introducción al *Backend*

- El *backend* incluirá, al menos:
  1. Los servidores de apoyo al *frontend*
  2. Los servidores REST, gRPC y Kafka de apoyo (internos)
  3. El servidor externo de conversación
  4. Un cliente de prueba (`TestClient`)
  5. Sistema de Monitorización (Prometheus)

Servicios Internos Java REST

- Se implementarán **uno o varios** servicios REST internos en Java que ofrecerán toda la información necesitada por el *frontend* en forma de URLs de recursos REST
- Estos servicios estarán protegidos y requerirán un **Token JWT** válido en la cabecera `Authorization`
- Los servicios encargados del *frontend* realizarán llamadas REST a este servicio para crear, consultar, eliminar usuarios, gestionar conversaciones, y hacer preguntas en *prompt*

Servicios Internos Java REST II

- Serán servicios REST implementados en Java que exportarán un conjunto de recursos:

  `/u/ID`

  Donde `ID` es el ID de un usuario, ofrecerá la información de un usuario
- Al menos, de cada **usuario** se tiene que almacenar:
  - su ID,
  - su nombre,
  - la contraseña (hash)
  - la lista de sus conversaciones
  - estadísticas de uso de los servicios,
  - etc.
- Cada usuario puede tener un conjunto de conversaciones

Servicios Internos Java REST III

- La siguiente URL se puede utilizar con `GET` para obtener la lista de conversaciones
- Si se utiliza con `POST`, crea una conversación:

```
/u/ID/dialogue
```

  En el cuerpo de la petición se incluirá el nombre de la conversación y otros parámetros que se necesiten:

```
{
  "dialogue_id": "DName",
  ...
}
```

- El resultado es una respuesta 201 (*Created*), donde la cabecera `Location` informa de la URL de la conversación

Servicios Internos Java REST IV

- La conversación tendrá pues la URL de recurso:

  ```
  /u/ID/dialogue/DName
  ```

  Donde `DName` indica el identificador de la conversación del usuario `ID`
- También se puede realizar la petición con el verbo `PUT`, donde la URL ya contiene el nombre de la conversación, y se puede enviar un contenido como en el caso de `POST`
- Si se hace `GET` a esta URL de recurso, se obtienen los datos de una conversación

Servicios Internos Java REST V

- Una conversación está compuesta por:
  - Su identificador ("`dialogue_id`"), `DName`
  - Un estado ("`status`"), que puede ser:
    - `READY` – La conversación puede aceptar un nuevo *prompt*
    - `BUSY` – La conversación está esperando una respuesta de 🦙Chat
    - `FINISHED` – La conversación ha terminado
  - Un *array* de objetos JSON, ("`dialogue`") donde cada elemento contiene:
    - Un *prompt*, ("`prompt`")
    - Una respuesta, ("`answer`")
    - Un *timestamp*, ("`timestamp`"), de cuando se produjo el *prompt*
  - Una `URL` para continuar la conversación ("`next`") a la que se le puede hacer `POST` con un nuevo *prompt*.
  - Una `URL` para terminar la conversación ("`end`"), a la que se le puede hacer `POST`

Servicios Internos Java REST VI

- Por ejemplo:

```
{
  "dialogue_id": "test",
  "status": "READY",
  "dialogue" :
        [ { "prompt": "hello",
            "answer": "hello, how are you?",
            "timestamp": 1234542323
          },
          ...
        ],
  "next": "/u/ID/dialogue/test/next/12323132122",
  "end": "/u/ID/dialogue/test/end"
}
```

- Si el estado no es "`READY`", no se permitirá el uso de `POST` a las URL de `next` ó `end`

Servicios Internos Java REST VII

- Una petición `POST` a la URL "`next`", si el estado de la conversación es `BUSY` ó `FINISHED`, devolverá el estado 204 (No content), y un mensaje indicando el error
- Si el estado es `READY`, la petición `POST` a la URL "`next`" será:

```
{
  "prompt": "...",
  "timestamp": 23123123
}
```

- El resultado llevará un código 201 (Accepted) e incluirá una etiqueta `Location:` con la URL de la conversación
- Esta petición causa una petición al servicio gRPC de *prompt* a 🦙Chat que se verá después
- Nótese que el hecho de enviar un *prompt* a este servicio hace que el estado de la conversación cambie (a `BUSY`), y también que se genere un nuevo token de la URL "`next`"

Servicio interno gRPC Java de *prompt* a 🦙Chat

- Se implementará, usando el API de gRPC, un servicio interno (en el *backend*) que permitirá hacer peticiones de respuestas a *prompts*
- Se deberá diseñar un API gRPC (tipos de datos, servicios y operaciones)
- **Importante:** La autenticación entre servicios internos también debe validarse. Si fuera necesario pasar contexto de usuario, el JWT viajará en los **metadatos** de la llamada gRPC.
- Debido a que la respuesta **puede tardar un tiempo considerable**, se deberá tener en cuenta la naturaleza *asíncrona* del servicio
- Este servicio gRPC a su vez hará uso del servicio 🦙Chat proporcionado

Servicio proporcionado 🦙Chat

- El servicio 🦙Chat se ofrece para responder a peticiones de *prompt*
- Es un servicio REST implementado en Python+Flask que hace uso de un modelo Llama 2
- El servicio exporta un recurso: `/prompt`, que se podrá usar con `POST`
- Si el servicio no está inicializado, devolverá una respuesta 102 (Processing)
- Si está inicializado, la respuesta tendrá un código 202 (Accepted) y contendrá la cabecera "`Location`" con la URL para obtener la respuesta
- La URL "`/healthcheck`" se puede usar para saber si el servicio está listo para su uso.
- Se ofrece el contenedor `dsevilla/ssdd-llamachat:1.0` y el dummy `dsevilla/ssdd-llamachat-dummy:1.0` (pausa artificial de 5s)

Servicio externo REST de *prompt*

- El servidor REST externo exportará el **puerto 8180**
- Este servidor REST externo hará uso del servidor interno gRPC visto antes y podrá acceder a los *prompts* y generar respuestas
- El servidor podrá servir todas las URLs de *prompt* (`/u/.../dialogue`), y permitir crear, eliminar y continuar con conversaciones
- Como es un servidor externo, el servicio funcionará **autenticado** requiriendo obligatoriamente un **JWT válido**.
- **OJO: El proyecto de ejemplo no incluye este servidor, hay que crearlo**

Seguridad y Autenticación

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

Autenticación con JWT

- Toda la seguridad del sistema se basará en **JSON Web Tokens (JWT)**.
- No se utilizarán mecanismos de estado en servidor (sesiones en memoria), garantizando que el sistema sea *stateless* y escalable.
- **Flujo de Autenticación:**
  1. El cliente (Frontend Web o Cliente Externo) envía credenciales (usuario/pass) al Backend.
  2. El Backend valida en BBDD y genera un JWT firmado con una clave secreta (HMAC SHA256).
  3. El JWT se devuelve al cliente.

Autenticación con JWT II

- **Uso del Token:**
  - El cliente debe almacenar el token.
  - En **cada petición** a un recurso protegido (REST), el cliente debe incluir la cabecera:

    ```
    Authorization: Bearer <token_jwt>
    ```

  - Para peticiones **gRPC**, el token se enviará como metadato (Metadata/Context).
- El Backend verificará la firma y expiración del token en cada petición antes de procesarla (usando Filtros/Interceptores).

Monitorización

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

Monitorización con Prometheus

- El sistema distribuido debe ser observable. Para ello se utilizará **Prometheus**.
- Se desplegará un contenedor oficial de Prometheus en el `docker-compose`.
- **Instrumentación de servicios:**
  - Tanto el Frontend (Flask) como el Backend (Java) deben exponer un endpoint público `/metrics`.
  - Prometheus consultará periódicamente estos endpoints (modelo *Pull*) para recolectar métricas.

Monitorización con Prometheus II

- **Backend (Java):**
  - Se usará la librería cliente de Prometheus para Java.
  - Debe exponer métricas de la JVM (memoria, hilos) y métricas HTTP.
- **Frontend (Flask):**
  - Se usará la librería `prometheus-flask-exporter`.
  - Debe exponer tiempos de respuesta y códigos de estado.
- Se valorará la creación de alguna métrica de negocio personalizada (ej: número de conversaciones iniciadas).

Nivel de Datos

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

Introducción al nivel de datos

- Se implementarán uno o varios contenedores con bases de datos
- Se puede elegir servidores SQL o NoSQL
- Se valorará que el código sea lo más independiente posible de la base de datos utilizada (patrón DAO)
- Las bases de datos guardarán toda la información necesaria: usuarios, **hash de contraseñas**, conversaciones y estadísticas.

Cliente de prueba

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

Cliente de prueba

- Se construirá un **Cliente de prueba** (`TestClient`)
- El cliente servirá para implementar varias pruebas sobre el *backend* usando los servicios REST/gRPC
- El cliente deberá ser capaz de **autenticarse, obtener un JWT** y usarlo para las peticiones subsiguientes.
- El cliente de prueba debe simular, al menos:
  1. El registro de un usuario
  2. La creación de una conversación y hacer peticiones de *prompt*, eliminar una conversación, etc.
  3. Mostrado de la conversación

Testing End-to-End

1. Introducción
2. Especificación del proyecto
   - Frontend
   - Backend
   - Seguridad y Autenticación
   - Monitorización
   - Nivel de Datos
   - Cliente de prueba
   - Testing End-to-End

Validación Automática (Selenium)

- Además del cliente de prueba (que valida el API), se requiere implementar **Tests End-to-End (E2E)** usando **Selenium WebDriver**.
- **Objetivo:** Validar que la interfaz web funciona correctamente con la asincronía (JavaScript/Polling).
- Se deberá entregar un script de test (en Python o Java) que realice automáticamente el siguiente flujo:
  1. Abrir el navegador y cargar el Frontend.
  2. Realizar login con un usuario válido.
  3. Navegar a la pantalla de chat.
  4. Enviar un *prompt* al sistema.
  5. **Esperar explícitamente** (Explicit Wait) a que aparezca la respuesta de la IA en el DOM.
  6. Validar que el texto de respuesta no está vacío.
- Este test debe ejecutarse contra el entorno desplegado en Docker.